



THE UNIVERSITY OF
SYDNEY

Room Number _____
Seat Number _____
Student Number _____

ANONYMOUSLY MARKED

(Please do not write your name on this exam paper)

CONFIDENTIAL EXAM PAPER

This paper is not to be removed from the exam venue

Computer Science

EXAMINATION

Semester 1 - Practice, 2025

COMP9123 Data Structures and Algorithms

For Examiner Use Only

EXAM WRITING TIME: 2 hours
READING TIME: 10 minutes

EXAM CONDITIONS:

This is a RESTRICTED OPEN book exam - specified materials permitted

MATERIALS PERMITTED IN THE EXAM VENUE:

(No electronic aids are permitted e.g. laptops, phones)

Calculator - non-programmable

One A4 sheet of handwritten and/or typed notes, double-sided

MATERIALS TO BE SUPPLIED TO STUDENTS:

This exam paper booklet.

INSTRUCTIONS TO STUDENTS:

Write your student ID at the top right of this front page.

Write your answers in the spaces provided in the exam paper booklet using blue or black ink. DO NOT ATTACH EXTRA PAGES TO THIS EXAM BOOKLET. If you run out of space in the space provided right after each problem statement, continue your answers on pages 20-26.

This paper contains four (04) questions, each worth 15 marks.
Answer ALL Questions.

Q	Mark
1	
2	
3	
4	

Total _____

Please tick the box to confirm that your examination paper is complete. ☐

Problem 1. (15 Marks)

Consider the *Dynamic Matrix* ADT for representing an matrix $A = \{a_{i,j}\}_{i,j=1}^n$ that supports the following operations:

- **CREATE()**: creates a 1×1 matrix where $a_{1,1} = 0$.
- **SET/GET(i, j)**: set or get the value of the entry $a_{i,j}$.
- **INCREASE-SIZE**: If the current size of the matrix is $n \times n$, increase it to $n + 1 \times n + 1$ such that the new entries are set of 0. In other words, A becomes A' such that $a'_{i,j} = a_{i,j}$ if $1 \leq i, j \leq n$, and $a'_{i,j} = 0$ otherwise.

Your task is to come up with a data structure implementation for the Dynamic Matrix ADT that uses $O(n^2)$ space, where n is the size of the matrix, and **CREATE**, **SET**, **GET** take $O(1)$ and **INCREASE-SIZE** takes $O(n)$ time.

- a) Describe your data structure implementation in plain English.
(07 Marks)

(Solution to Problem 1 (a) continues here)



- b) Write structured pseudo-code for the algorithm you described in part (a).
(05 Marks)



- c) Analyze the time and space complexity of your data structure using Big-O notation.
(03 Marks)

A large empty rectangular box with a thin black border, intended for the student to write their analysis of time and space complexity. A large, light gray watermark reading "Practice Exam" is diagonally across the box.

Problem 2. (15 Marks)

Let G be a connected undirected graph on n vertices. We say that two distinct spanning trees T and S of G are *one swap away* from each other if $|T \cap S| = n - 2$; that is, T and S differ in only one edge.

For two distinct spanning trees T and S we say that $R_1, R_2, \dots, R_k$ form a *swapping sequence* from T to S if:

1. $R_1 = T$,
2. $R_k = S$, and
3. for any $1 \leq i < k$, the trees R_i and R_{i+1} are one swap away from each other

Your task is to design a polynomial time algorithm that given G and two spanning trees T and S of G , constructs a minimum length swapping sequence.

- a) Describe your algorithm in plain English.
(07 Marks)

(Solution to Problem 2 (a) continues here)



- b) Prove the correctness of your algorithm.
(05 Marks)



A large empty rectangular box intended for the student to write their proof. A large, light gray watermark reading "Practice Exam" is diagonally across the box.

(Solution to Problem 2 (b) continues here)



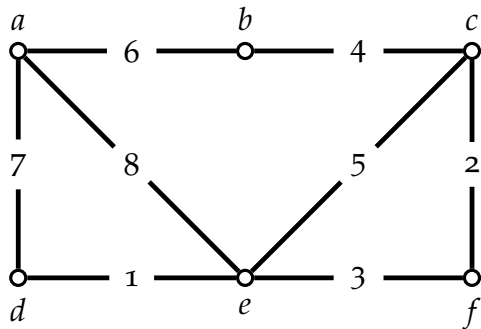
- c) Analyse the time complexity of your algorithm.
(03 Marks)



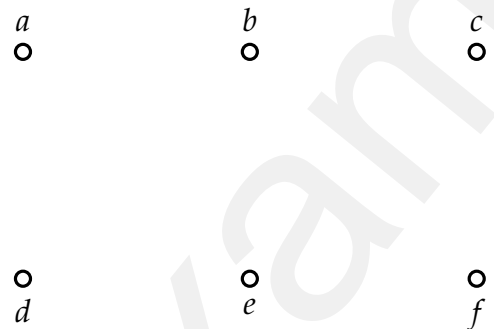
A large empty rectangular box for writing the answer. A large, light gray watermark reading "Practice Exam" is diagonally across the box.

Problem 3. (15 Marks)

- a) Consider the weighted graph G on the left. Draw the minimum spanning tree (MST) of G on the right. You do **not** have to explain your answer.
(04 Marks)

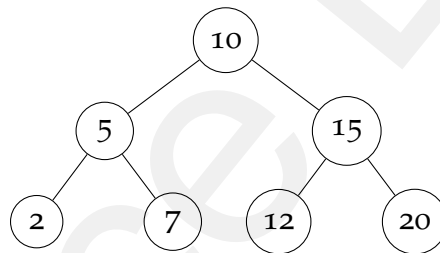


Graph G



MST(G)

- b) Consider the following Binary Search Tree (BST):



- Write the pre-order and post-order traversals of the BST.
(02 Marks)
- Delete the node with key 15 from the tree. Draw the resulting BST.
(02 Marks)

(Solution to Problem 3 (b) continues here)



- c) You are designing a data storage system that needs to support real-time ranking of numeric user scores. Users frequently insert new scores into the system, and your goal is to efficiently track and query the median score at any time. To support this, you are asked to design a data structure that allows:

- Inserting a new score, and
- Querying the current median score

both in $O(\log n)$ time per operation.

Describe a solution based on an augmented AVL tree. Clearly explain in plain English:

- What additional information must be stored in each node,
- How it is maintained during insertions and rotations,
- How it allows efficient computation of the median without sorting all scores.



(Solution to Problem 3 (c) continues here)



Problem 4. (15 Marks)

Let A be an array containing n distinct integers, representing locations of wireless transponders. Each transponder has an identical range d . Two wireless transponders i and j can communicate with each other if and only if $|A[i] - A[j]| \leq d$, i.e., if the absolute difference between their locations is at most the distance they can cover. Recall that $|x|$ equals x when $x \geq 0$ and $|x|$ equals $-x$ if $x < 0$. You need to determine how many pairs of transponders can communicate with each other. Your task is to compute the number of pairs of wireless transponders that can communicate with each other.

Example:

When $A = [4, 2, 12, 7]$ and $d = 3$, you should return 2, since $|4 - 2| = 2 \leq d$ and $|4 - 7| = 3 \leq d$ and no other pair is at distance at most d from each other.

- a) Design a divide-and-conquer algorithm to solve this problem in $O(n \log(n))$ time. Explain your algorithm clearly in plain English.
(08 Marks)

(Solution to Problem 4 (a) continues here)



- b) Argue the correctness of your algorithm.
(05 Marks)



A large empty rectangular box for writing the answer. A large, light gray watermark reading "Practice Exam" is diagonally across the box.

(Solution to Problem 4 (b) continues here)



- c) Analyse the time complexity of your algorithm. Express your answer using Big-O notation.
(02 Marks)



THIS AND FOLLOWING PAGES LEFT INTENTIONALLY BLANK.

Use this and following pages for additional writing space if you run out of space in an answer area. If you want any writing on this page or following pages to be marked, you **MUST** write “see additional pages” in the relevant answer area and clearly indicate the question number you are answering on the relevant page. Any writing outside of this booklet will not be examined.



(use the rest of the space of rough work)

A large rectangular box intended for rough work. It contains a diagonal watermark that reads "Practice Exam" in a light gray, sans-serif font, oriented from the bottom-left towards the top-right.

(use the rest of the space of rough work)

A large rectangular box intended for rough work. It contains a large, light gray, diagonal watermark that reads "Practice Exam".

(use the rest of the space of rough work)

A large rectangular box intended for rough work. It contains a large, light gray diagonal watermark that reads "Practice Exam".

(use the rest of the space of rough work)

A large rectangular box intended for rough work. It contains a large, light gray, diagonal watermark that reads "Practice Exam".

(use the rest of the space of rough work)

A large rectangular box intended for rough work. It contains a large, light gray diagonal watermark that reads "Practice Exam".

(use the rest of the space of rough work)

A large rectangular box intended for rough work. It contains a diagonal watermark that reads "Practice Exam" in a light gray, sans-serif font, oriented from the bottom-left towards the top-right.